



Ingenuity of scratch programming on reflective thinking towards problem solving and computational thinking

Semirhan Gökçe¹ · Arzu Aydoğan Yenmez¹

Received: 28 May 2022 / Accepted: 28 September 2022

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract

Individuals learn to develop problem solving strategies and make connections between their mathematical ideas while programming, so they have the opportunity to improve their thinking skills. Scratch provides an environment to experience problem scenarios and encourages them to act out imagination while having fun. The purpose of the study is to investigate the development of reflective thinking skills towards problem solving and computational thinking of elementary school students based on their Scratch instruction. The study used a sequential exploratory design as a mixed method approach with 524 students and 8 mathematics teachers. The results of the study revealed that Scratch significantly strengthened students' reflective thinking skills for problem solving and computational thinking. Based on the results, the reflective thinking towards problem solving and computational thinking did not vary by gender. In addition, Scratch instruction led to positive reflections in the mathematical learning environment.

Keywords Computational thinking · Problem solving · Reflective thinking · Scratch · Thinking skills

1 Introduction

Industry 4.0, brought about by the rapid technological change of recent years, shows that innovations such as mobile devices and Internet technologies have begun to profoundly affect our daily lives. The educational system of industrial society has evolved over the past two centuries, and attempts to develop an educational system

✉ Semirhan Gökçe
semirhan@gmail.com

¹ Department of Mathematics and Science Education, Niğde Ömer Halisdemir University, Niğde, Turkey

with 21st-century skills have intensified. Science, technology, engineering and mathematics (STEM) involves interdisciplinary, concrete, application-oriented practices aimed at solving problems arising from the events of everyday life (coding in various dimensions in different disciplines, robotics and design activities for smart devices). The changes brought about by Industry 4.0 often lead to debates the skills people can have in the 21st century. At this time, more studies have been done on 21st century skills as people develop a better understanding of the rapidly changing world. Computational thinking and coding were among these skills that individuals should have, according to studies published by International Education Technology Society (ISTE) in 2007 and 2016.

The individuals learn to make connections between ideas and design learning strategies and projects related to problem solving while coding, which will enhance their mathematical and analytical thinking skills (such as variable and conditional expressions). Along with computational thinking, communication, relationship skills, and collaboration are all skills that can be taught. Coding can embody abstract concepts, combined with skills such as creativity, teamwork and problem solving. These can also make coding an effective tool for gaining computational thinking skills. These skills are not only needed by computer programmers, but also by people from all fields, professions, and occupations (Resnick, 2013; Wei et al., 2021; Wing, 2006).

Mathematics education has undergone significant changes so that the teaching styles have been reorganized in accordance with learning objectives (Chevallard & Bosch, 2020; Olkun & Toluk-Uçar, 2006). It is not enough for a student to have knowledge; it is also important for them to be able to develop cause-and-effect relationships (Brewer, 2007; Kuo, Barnes & Jordan, 2019; Lind 1998), as well as be creative and adaptable to new ideas (Rowe, 2007). Instead of teaching mathematical knowledge, the aim is to educate individuals who can apply what they have learned (Demirel & Yağcı, 2012; Witrock, 1978), who can solve problems (Clements & Gullo, 1984; Kenedi et al., 2019; Korkmaz et al., 2015), who can communicate well, and who have a positive attitude towards mathematics (Davadas & Lay, 2020; Tutak, 2008). At this point, mathematics teaching can be supported with technological materials that have an active role in concretizing the abstract concepts (Celen, 2020; Gelibolu, 2009; Kaput, 1999; Kieran, 1992; Taşlibeyaz, 2010). Mathematical software enables modeling (Aksakal et al., 2015) and understanding of the problem solving process (Clements & Gullo, 1984; Korkmaz et al., 2015), through the concretization of numerical, algebraic and graphical representations (Azuma, 1997; Zengin, 2019).

Programming instruction can help students acquire basic skills, such as problem solving, by developing their metacognitive skills (Mangaroska et al., 2022; Zuckerman et al., 2009). Students who experience mathematics with coding develop problem solving skills, visual perspectives and algorithmic thinking skills (Sun et al., 2021; Taylor et al., 2010; Wing, 2006; Domínguez et al., 2013) describe algorithm as a plan that specifies what the computer should do at each step to get to the desired result. Algorithms are used in programming design and consist of many steps. Therefore, the first thing before creating a program is to work on its algorithm. Studies show that students who receive programming instruction have higher thinking, creativity and problem solving skills compared to students who do not receive instruction (Clements & Gullo, 1984; Liu et al., 2021). For example, students who learn

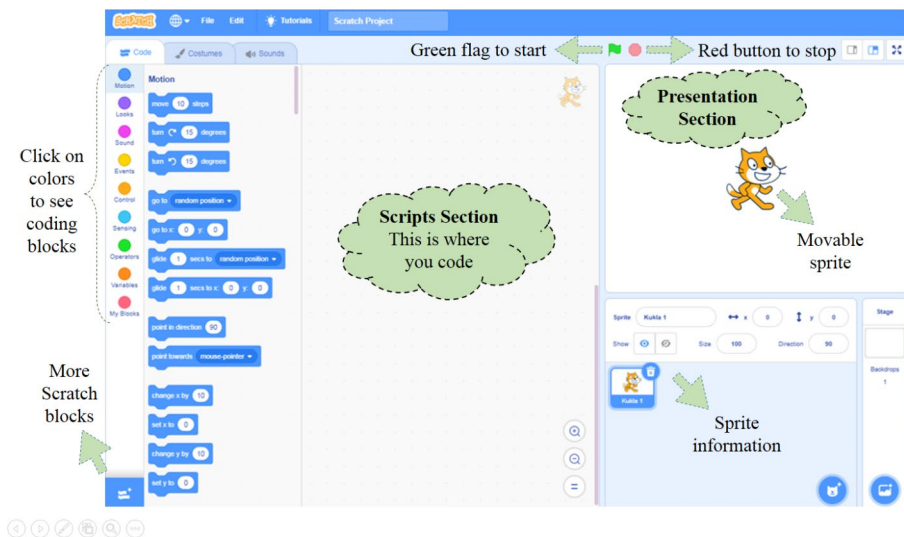


Fig. 1 A screenshot of Scratch interface

mathematics with software can look at problems from different perspectives, which leads to more innovative learning (Korkmaz et al., 2015). As a result, coding is considered important not only for programmers, but for all students. In the process of teaching mathematics with coding, the problem is first analyzed, then algorithms are developed, next the algorithm is evaluated, and if it works correctly, the algorithm is coded (Fesakis & Serafeim, 2009). These steps help students learn more efficiently in a collaborative environment through the use of visual applications (Garner, 2003; Hsu et al., 2018).

Some initiatives are emerging in society to enable new generations to write or design, create and invent on a larger scale. For example, visual block-based programming platforms such as Alice, Kodu Game Lab, Code.org and Scratch provide environments where children can develop their own programs, games and animations without having to learn the complex code structures of traditional programming languages (Resnick et al., 2009; Weng et al., 2022). Scratch is one such coding environment that could be used in the mathematics classroom to help students visualize their ideas while having fun (Gonzalez, 2012; Rodríguez-Martínez et al., 2020). A screenshot of Scratch can be seen in Fig. 1.

Scratch is also mentioned as an environment for an “educational game” in the classroom (Alakoç, 2003). Games are the primary activities that engage children and teach while entertaining them (Kalogiannakis et al., 2018; Wells et al., 2008). An educational game helps children solve problems in daily life so that new tasks are introduced and as children continue to solve these problems, they become problem solvers. Therefore, a game is an effective learning process for children and a motivating factor for teaching. Using games to present the topic arouses their interest in the lesson and makes it easier for them to understand it. We can gain target behaviors by converting educational games into computer-based lessons (Kukul, 2013).

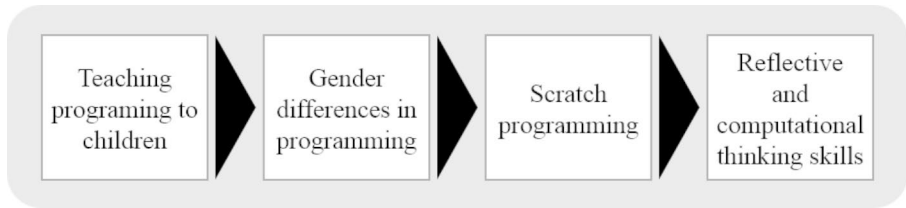


Fig. 2 Sections of theoretical framework

As the types of games for boys and girls differ, the considerations of games on the education/teaching process attract attention. Researchers have also addressed the issue of gender parity in computer use since the dawn of the personal computer. It is stated that if males and females participate differently in computer learning settings, this could lead to inequalities in cognitive skills and access to the labor market (Ferracane, 2020; Linn, 1985). In Linn's study, it was found that once girls and boys participate in the class, they achieve equivalent levels of proficiency in programming, but that girls are less likely to enroll. According to some other studies, the female students are significantly less confident in their programming abilities than male students (Alvarado, Lee & Gillespie, 2014; Carter & Jenkins, 1999).

Based on these findings, the purpose of the study is to investigate the development of reflective thinking skills towards problem solving and computational thinking of elementary school students based on Scratch instruction. The main motivation of this study is to contribute to the literature in this context by conducting a study that deals with the above-mentioned variables together. The research problems of the study are as follows:

RQ1: Does Scratch instruction provide a significant difference on reflective thinking towards problem solving and computational thinking scores?

RQ2: Is there a gender difference on reflective thinking skills towards problem solving and computational thinking skills of elementary school students?

RQ3: What are the opinions of the students taking Scratch instruction about the mathematical learning process?

RQ4: What are the opinions of mathematics teachers about the changes in the mathematical learning process of students who take Scratch instruction?

1.1 Theoretical Framework

This section provides information regarding the key concepts and the relations among them. Since the purpose of the study is to investigate the development of reflective thinking skills towards problem solving and computational thinking of elementary school students based on Scratch instruction, the order of the concepts from general to specific is given in the Fig. 2 below.

1.1.1 Teaching programming to children

Programming makes our lives easier, so it is no surprise that teaching programming is one of the trend topics in schools. Basic versions of college-level programming

courses are also offered in elementary schools and even kindergartens. The prediction that programming education will be one of the skills of the 21st century, along with the introduction of visual programming languages, has sparked a worldwide interest and trend in programming education. Computer Science For All (CSForALL) is a movement in the United States that aims to increase computer science skills for all students, from kindergarten to high school (Margolis & Goode, 2016). Primary school students benefit from programming exposure in terms of cognitive development (Atmatzidou & Demetriadis, 2016; Lewis, 2010) and their perceptions toward programming (Kalelioğlu & Gülbahar, 2014a; Lewis, 2010). Studies conducted in recent years have shown that children who learn programming improve their ability to produce solutions to the problems they encounter. It is claimed that children, who can find their mistakes more easily thanks to their coding skills, have increased their ability to evaluate results (Arfé et al., 2020; Coravu, Marian, & Ganea, 2015; Çiftçi & Bildiren 2020; Erol & Çırak, 2022; Resnick & Silverman, 2005).

1.1.2 Gender differences in programming

Human lives are intertwined with computer science (CS) and information technology (IT). The technological revolution has benefited both men and women. However, females have a smaller percentage of education and limited options in career choice in several parts of the world (Mayer-Smith et al., 2000). Cultural perceptions, socialisation of gender roles, lack of female participation in computer science, and the way computers are taught and practised in schools are all factors that contribute to gender inequality (Shashaani & Khalili, 2001). Females have a lower interest in computer science and technical disciplines. There are studies that show that changing the environment can help females perform better (Shashaani & Khalili, 2001). On the other hand, the Organisation for Economic Cooperation and Development [OECD] (2011) found that female students outperformed male students in their studies measuring digital literacy among 15-year-old students in 19 countries. Similarly, female students had significantly higher scores than male students in programming and digital literacy evaluation studies conducted by National Assessment of Educational Progress (NAEP) at the 8th grade level in the United States (Altinok et al., 2018).

The problem solving and programming unit consists of coding and programming titles as content, and male students are more enthusiastic and interested in computing and programming (Du et al., 2016; Lockheed, 1985). However, some studies show that female students have become more interested in programming and coding (Colley & Comber, 2003; Fraillon et al., 2014). According to Hsu (2014), students' level of understanding of programming frameworks did not differ by gender in Scratch programming at the primary school level but female students learned algorithmic concepts better. Durak & Guyer (2019) used Scratch to evaluate the effectiveness of programming education in special education at the primary school level, and discovered a substantial difference in Scratch skills in favor of female students at the end of 15 weeks of programming instruction. In the observations, female students in both experimental and control groups were found to be more active and eager participants in the lessons. Male students, on the other hand, were observed to spend their time on extracurricular topics and constantly playing computer games with each other

(Quaiser-Pohl et al., 2006) rather than on their assigned tasks. This also supports the quantitative data, as female students asked their teachers more questions about their daily routines as well as the projects they worked on during recess and extracurricular activities. In addition, it was observed that female students in all groups had more positive attitudes towards programming than male students. In a study conducted by Akşit (2018) with middle school science students, it was found that Scratch and activities aimed at improving computational thinking skills ended up with a significant difference in students' attitudes toward programming in favor of female students.

1.1.3 Scratch programming

Scratch is a free programming language developed by Massachusetts Institute of Technology (MIT) Media Lab in 2007 to create interactive stories, games, and animations. Since then, Scratch has been translated into more than 50 languages and used in more than 150 countries. Thanks to this feature, projects can be created and shared by users in this environment (Scratch About, 2021). Scratch was defined as a visual, block-based programming language that facilitates media editing for novice programmers (Maloney et al., 2008). It has an interface that allows users to write programs by placing code blocks using the drag-and-drop system without having to write the codes themselves (AbdulSamad & Romli, 2022; Sivilotti & Laugel, 2008). Many users learn Scratch by dragging and dropping blocks of code, by trial and error, or by discovering and investigating coding in existing projects. Scratch's programming environment provides with ease of access, vitality and discoverability with a single-window user interface, ease of control through visual output of code, coding without error messages, concreteness of data, and minimization of code blocks (Maloney et al., 2010). Scratch provides a more memorable, meaningful and social environment than other programming environments.

Users from all age groups have practiced Scratch, but those aged 8 to 16 are the most involved group (Scratch Stats, 2021) that corresponds from primary school to high school. Scratch can be used in the projects of different disciplines such as mathematics, science and foreign language education (Scratch About, 2021). Students can learn more easily by using the app to visualize problems in their minds. Instructors concluded that Scratch makes programming classes more enjoyable and inspiring, easier to understand, allows for creativity, and facilitate interdisciplinary learning after Scratch training was offered as part of the computer literacy course (Flanagan, 2015). In addition, according to a study conducted on primary school students by Kobsiripat (2015), Scratch has a positive impact on creative thinking skills. In the study, which examined 32 articles about Scratch, the prominent topics related to Scratch were creative thinking, logical thinking, problem solving, algorithm and programming instruction, simulation development, game programming, comparison with other languages, use in other courses, students' opinions about programming and introduction of Scratch (Çatlak et al., 2015). Moreover, this study found that Scratch provides an efficient environment for teaching programming, it makes programming exciting and fun, and that starting with Scratch increases interest and motivation in the classroom.

1.1.4 Reflective and computational thinking skills

The two types of thinking skills are basic and higher-order thinking skills. Lewis & Smith (1993) characterized higher-order thinking skills as a way of thinking in which a person expands knowledge by retrieving and storing information in memory to achieve a goal, and reorganizing it as needed. The researchers emphasized that higher order thinking involves creative thinking, problem solving, critical thinking, and decision making. In this context, the concept of reflective thinking, which includes critical thinking and creative thinking and problem solving skills, appeared two centuries ago. Wilhelm von Humboldt was the first to use the term “reflective learning,” and emphasized the importance of not only what to learn but also how to learn (Fichtner, 2005). John Dewey’s approach of learning by doing in turn laid the foundation for reflection. Dewey (1933) argued that the most important need of society was to teach students how to represent what they had learned in school and described reflective thinking as an active and deliberate process involving the stringing together of coherent ideas through reasoning and including knowledge and values.

Reflective thinking, according to Üstünoğlu (2006), is defined as being open to one’s own and others’ ideas, reflecting one’s thoughts easily, behaving honestly, and anticipating the future. Cengiz (2014), on the other hand, understands reflective thinking as interacting with peers and teachers to create new perspectives and make self-evaluation in order to achieve learning goals, maintain motivation, provide deep understanding, use appropriate learning strategies, and improve the learning process and performance. “Reflective thinking is crucial because it guides the student toward learning goals, makes him think about the technique he uses, helps him build the ability to take responsibility and solve difficulties, and also guides him to gain self-evaluation skills,” (p.13) according to Ünver (2011). In mathematics class, it is crucial to use problem solving skills, drawing diagrams, and understanding the problem. In addition, students who have cognitive skills such as reflective thinking are usually one step ahead of the class and actively participate in class.

Individuals should think differently, look at problems from different angles, and adjust their problem-solving strategies as the new world order changes what is expected of them. For this reason, all members of society should develop problem-solving and reflective thinking strategies in all extraordinary situations they encounter in their daily lives (Antonio, 2020; Göksün & Kurt, 2017). In this regard, one of the most important skills that individuals should have, especially in recent years, is critical thinking for problem solving (Shanta & Wells, 2020; Sholihah & Lastariwati, 2020). This condition can be attributed to the advantages of reflective thinking in the problem solving process. Since reflective thinking arises with the perception of a problem (Shermis, 1992), it can be said that reflection can be best observed in the problem solving process. Moreover, according to Kızılkaya & Aşkar (2009), reflective thinking can be best found in the problem solving process and consists predominantly of reflective habits such as questioning (looking for answers to problems), reasoning (looking for cause-and-effect relationships), and evaluating (taking a step back and thinking about how problems are actually solved).

Programming is similar to the problem solving process, in that it often uses similar steps. Programming can also improve the efficiency of solving problem and the abil-

ity to analyze, because it is necessary to divide the problem into subsets and to create a basic solution that can be generalized (Saeli et al., 2011). Consequently, developing problem-solving skills in a variety of domains through programming is also a priority, and the evidence for the role of programming in developing problem-solving skills is encouraging (Dasso et al., 2005; Kim et al., 2019; Papert, 1993). Through programming, the concepts of certain disciplines can be more easily interpreted and put into practice. For example, in mathematics, concepts such as variables and functions can be better understood through programming. Therefore, this ongoing process leads students to manage their programming processes, question their decisions and actions and explore alternative solutions to develop the quality of their programs (Havenga et al., 2013).

According to ISTE (2015), computational thinking is a problem solving approach that enhances the combination of technology and thinking. According to Wing (2008), computational thinking is a form of analytical thinking. Many people associate computational thinking with problem solving, but the term has been broadened to include basic computer science concepts such as abstraction, separation (analysis), pattern generation, visualization, problem solving, and algorithmic thinking (Curzon, 2015; Furber, 2012) defines computational thinking as the process of recognizing various aspects of information processing in the world around us, the use of computer science tools and techniques to understand and reason about natural and artificial systems and processes. Bråting and Kilhamn (2021) focused on how the implementation of Lightbot and Scratch programming in school mathematics interacts with algebraic and computational thinking. They focused on three examples of programming activities for school mathematics and compared semiotic representations of programming languages with algebraic notation. According to Wing (2006), computational thinking is an area of thinking skills that includes problem solving, system design and understanding human behavior based on computer science concepts. Wing also asserts that this is a skill that everyone should have, not just those who work in the field of computer science. According to ISTE (2016), computational thinking is a key concept that people will need today and tomorrow, and it is described as a problem solving approach that empowers technology and thinking based on it. Wing (2014) considers computational thinking as a basic skill like mathematics, reading and writing skills, and states that individuals should have and develop their computational thinking skills.

Computational thinking plays an active role not only in computer engineers but also in improving the ability of all individuals to solve the problems they encounter in their daily lives. The traces of computational thinking, which is counted to be among the 21st century skills and should be among the future skills of the workforce, date back to the study of Seymour Papert. According to Papert (1980), the individuals first have to find a solution to the given problem in programming. Then, they have to find a way to communicate with the computer about that solution. According to Wing (2006), computational thinking should be one of the basic skills that should be acquired in curricula. Wing's study led to a renewed interest in increasing computational thinking at the K-12 level. Computational thinking is defined as the process of thinking to formulate and solve problems using computers. It is fair to say that, computational thinking has moved up the agenda since that date. For example, in the

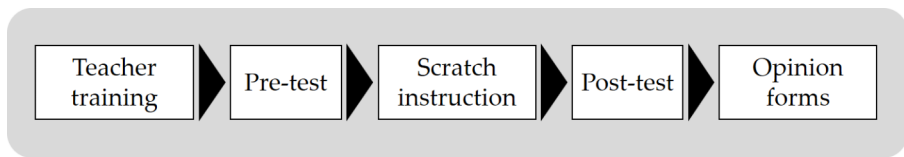


Fig. 3 Procedure of the study

Standards for Students report although the sub-headings of computational thinking, such as critical thinking, problem solving, decision making, creativity and innovation, are listed separately, computational thinking is not included as a concept (ISTE, 2007). However, in 2011, the ISTE produced the report “Operational Definition of Computational Thinking” (ISTE, 2011). In order to define the link between programming education and computational thinking, the idea that programming education is not synonymous has been emphasized by many researchers and scholars working in this field, and that programming education contributes to individuals’ computational thinking skills (Allsop, 2015; Angeli et al., 2016; Kalelioğlu & Gülbahar, 2014b; ScratchEd Team, 2011; Settle & Perkovic, 2010; Wing, 2006).

2 Method

The sequential exploratory design as a mixed method approach is used in this study and this pattern is realized in two stages. The procedure of this study is indicated in Fig. 3.

Based on the research questions, the collection and analysis of quantitative data came first, followed by the collection and analysis of qualitative data. The importance of qualitative results in explaining quantitative results was emphasized, and quantitative results are predominant in this design (Creswell & Clark, 2018).

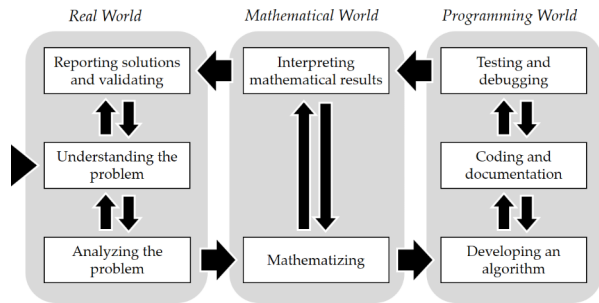
2.1 Participants

The participants in this study were 524 elementary school students from grades 5 and 6 (245 girls and 279 boys) and 8 mathematics teachers who supervised their mathematics instruction. The age of the students ranged from 10 to 12 years old. The school, which was randomly selected from a list of schools, has been located in the center of a city in Central Anatolian Region of Turkey. The students had no prior knowledge or experience with programming and only learned about Scratch during this study.

2.2 Scratch instruction

Scratch instruction were offered as part of the “Information Technologies and Software” course given in the elementary school program in grades 5 and 6. The content of the second semester of this course covers algorithms and block-based programming (i.e. Scratch). The grade 5 and 6 curricula both include similar learning outcomes for programming. For each grade level, the same information and communication tech-

Fig. 4 The cyclical structure of programming



nologies (ICT) teacher taught the course content and the math teachers observed the lessons so that they could focus on the students' work during class. Before Scratch instruction, the math teachers were informed about the content so that they could understand the process and analyze its impact on their math instruction. This teacher training was conducted by an expert programmer before the students started their Scratch instruction. This 8-week training explained that Scratch is a graphical programming language that can be used to create animations, design computer games, or tell interactive stories by using code block structures and combining various media tools such as images, sound, and music, and explained the basic functions of the program. During this time, the teachers made preliminary studies for the activities that the students would aim for in the course.

The students' Scratch lessons continued for one semester, i.e., a total of 1360 min, including two lessons per week (40 min per lesson) for 17 weeks. In order to determine what kind of cyclical process should be engaged in Scratch instruction, we examined the related theoretical and empirical literature, elementary mathematics curriculum and information technologies and software curriculum as well. The cyclical structure of programming formed in this context is shown in Fig. 4.

Given the functionality of this cyclical structure, the content covered in Scratch instruction is listed as follows: determining the steps of the problem-solving process, proposing different solutions to a problem, listing the necessary variables, constants, and operations to solve the problem, developing an algorithm to solve the problem, creating a flowchart of the algorithm, testing the solution and debugging an algorithm, recognizing the interface and functions of the block-based programming tool, creating the appropriate algorithm to achieve the goals presented in the block-based programming environment, developing an algorithm using a linear logic structure, and creating decision structures and loops (MoNE, 2018a).

The content of this course contains many similarities with the Turkish national mathematics curriculum (MoNE, 2018b). It corresponds to (1) mathematical literacy and basic competencies in science/technology and (2) digital literacy, which are considered within the basic competencies of mathematics curriculum. It also includes routine and non-routine problem-solving processes.

Scratch software features were introduced through activities that captured students' interest. Accordingly, games such as moving the snake, crossing the river, reaching the goal without being caught by moving obstacles, collecting milk boxes, creating a competition and hunting meteors, etc., were used to increase students' motivation to learn while developing thinking skills by creating a challenging problem situation. In



Fig. 5 An example code block used in Scratch instruction: Snake game

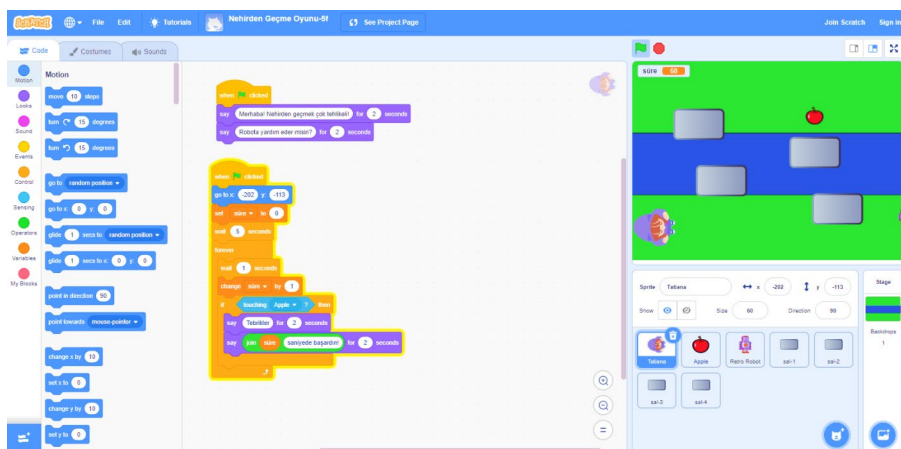


Fig. 6 An example code block used in Scratch instruction: Crossing the river

designing the games, the basic features such as variables, mathematical operations, conditional expressions and loops, were used along with the functions such as location, image and sound properties, sensor usage and messaging. Figure 5 shows the code blocks of the snake game.

The goal of the snake game is to control the snake using the arrow keys on the keyboard to reach all the targets. However, when the snake reaches the goals in this game, its movement speed increases and this situation makes it difficult to control the snake. With this activity, students could practice of using code structures such as variables, conditional expressions, loops, sensors, cloning, and message broadcasting. Another activity is related to crossing the river and Fig. 6 shows a screenshot.

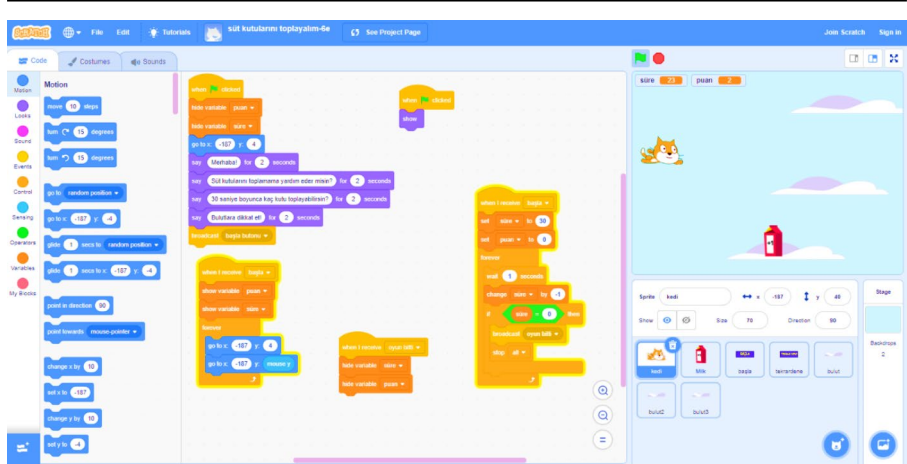


Fig. 7 An example code block used in Scratch instruction: Catching milk boxes

In the game “River Crossing”, the goal is for a character designed as a function of the keyboard keys to cross the river over the objects moving horizontally and vertically to reach the apple on the opposite shore. In creating the code structure of the game, it is necessary to display the message on the screen, start the game according to the given answer, change the position with the keyboard, check if the character touches the apple and log the time during this process. Students used coding features such as variable operations, conditional expressions, loops, and sensors to accomplish this goal.

In another activity that students used in the course, the cat character was made to catch milk boxes moving in the air parallel to the ground in a given time, which is shown in Fig. 7.

The character must do this without touching the clouds from different heights. In designing this game, students used basic programming operations such as defining variables, assigning values to variables, control structures, loops and sending messages, hiding and displaying objects depending on certain conditions.

The activities at the beginning of the instruction are more demonstration-oriented and involve a step-by-step approach (see the snake game in Fig. 5), while the other activities create learning environments in which students can express themselves in different perspectives (see the river crossing game and milk box game in Figs. 6 and 7, respectively). Each process is similar to mathematical problem solving. Routine problem solving involves the use of known or prescribed procedures to solve problems (Polya, 1973), such as step-by-step procedures, while non-routine problem solving encourages the use of different strategies and the exploration of relationships between variables (Elia et al., 2009), such as supporting different perspectives.

Students were asked to design a game in which a sprite is to achieve a goal, taking into account the code structures learned in the course. As an example, the development of catching the milk boxes game (see Fig. 7) is explained in Fig. 8 according to the cyclical structure of its' development.

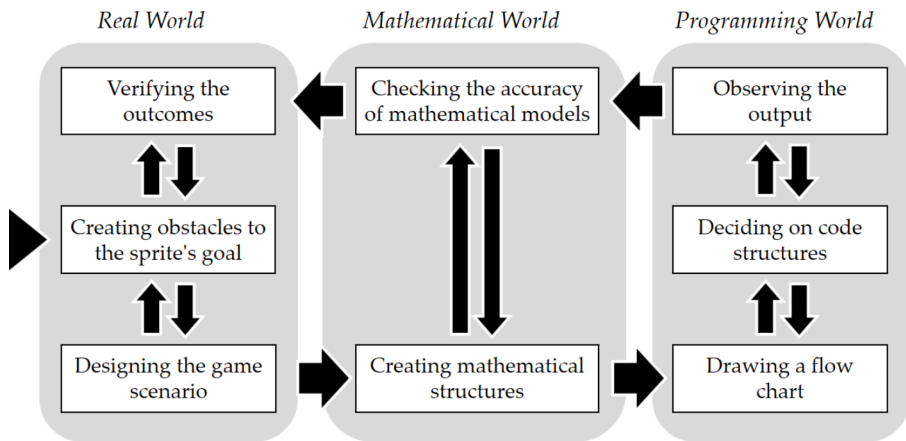


Fig. 8 The cyclical structure of the development of an example game

This game was created primarily based on a real event. Accordingly, students designed a scenario by relating the cat sprite in Scratch to the milk boxes. They used cloud imagery as obstacles and time as a constraint. Students also checked the compatibility of the designed game with real life.

In the mathematical world, students determined the necessary mathematical structures. Accordingly, they used translation in moving the sprites in the game, simple algebraic expressions in setting the time constraint, and random function in irregular formation of the obstacles. Students also verified that the mathematical structures in the game outputs were correct for each condition.

In the programming world, students first created a flowchart that considered all phases of the game, and they decided which code blocks to use. Students reviewed the outputs of the codes and made the necessary changes if they did not match the desired structure.

2.3 Data collection tools

The “Reflective Thinking Scale towards Problem Solving” and the “Computational Thinking Levels Scale” were applied to the students participating in the study. In addition, the written opinion forms were used to clarify the quantitative results of the scales.

2.3.1 Reflective thinking scale towards problem solving

The “reflective thinking scale towards problem solving” developed by Kızılkaya & Aşkar (2009) has 14 items in the form of 5-Likert type items. This scale was used as a pre-test and a post-test to measure the change in students’ reflective thinking skills toward problem solving. Two sample items are given as follows: (1) I ask questions to myself in order to understand why I could not solve a problem and (2) I wonder if I could find a better solution after solving the problem. If the mean of the scale is closer to 5, it means that the participant’s reflective thinking to solve the problem is high.

Table 1 Statistics for normality assumption

Scale	Type	Skewness	Kurtosis
Reflective thinking	Pre-	0.280	-0.354
	Post-	0.446	-0.733
Computational thinking	Pre-	-0.279	-0.359
	Post-	-0.401	-0.303

Otherwise, if the mean of the scale is closer to 1, it means that the participant's reflective thinking for problem solving is low. As a result of the analysis, the Cronbach's alpha coefficient of the scale was calculated as 0.919.

2.3.2 Computational thinking levels scale

In this study, the 22-item "computational thinking levels scale" developed by Korkmaz et al., (2015) was used to describe the students' ability level in computational thinking as a pretest and posttest. Two sample items are given as follows: (1) I make use of a systematic method while comparing the options at my hand and while reaching a decision and (2) I think that I learn better the instructions made with the help of mathematical symbols and concepts. The scale is of the 5-Likert type. When the mean of the scale approaches to 5, it is interpreted that the participant has high computational thinking ability and when the mean approaches to 1, it is interpreted that the participant has low computational thinking ability. Cronbach's alpha coefficient of this scale was found to be 0.855.

2.3.3 Written opinion forms

Students were given a written opinion form to complete in order to assess the impact of Scratch instruction on their mathematical learning. The forms had items that asked to what extent Scratch affected their concentration, participation, enjoyment, learning, and motivation in mathematics class. In these items, the options are: too much, much, enough, less and too less. There were also open-ended items in the forms to assess the different ways of thinking that are influenced by Scratch lessons. Similar questions were asked of teachers about how students evaluate their learning. For example, teachers were asked to what extent Scratch affects their students' concentration in mathematics class.

2.4 Data analysis

Repeated measures ANOVA is used to compare the means of reflective and computational thinking scales based on repeated observations for quantitative data analysis. One of the underlying assumptions of repeated measures ANOVA is normality, which stands for that each level of the independent variable needs to be approximately normally distributed. The skewness and kurtosis values are shown in Table 1.

Since skewness and kurtosis values up to the absolute value of 1 are considered as indicators of normality (Huck, 2012), the distribution of values is approximately normal. Another assumption is sphericity, which stands for equality of homogeneity

Table 2 Descriptive statistics of reflective thinking scale scores

Scale	Type	Gender	Mean	SD
Reflective thinking	Pre-test	Female	2.179	0.628
		Male	2.293	0.665
		Total	2.240	0.650
	Post-test	Female	2.676	0.756
		Male	2.738	0.732
		Total	2.709	0.743
	Computational thinking	Female	3.829	0.694
		Male	3.733	0.685
		Total	3.778	0.691
Computational thinking	Pre-test	Female	3.829	0.694
		Male	3.733	0.685
		Total	3.778	0.691
	Post-test	Female	4.117	0.600
		Male	4.028	0.581
		Total	4.070	0.591

of variances (Howell, 2002). The Levene's test showed that the variances of the variables were not significantly different with measurement repetition.

Content research was used to examine the opinions of teachers and students. The stages of developing a structure for content analysis, processing the data according to the thematic framework, identifying and interpreting the results were all considered when evaluating the content in the study. The code and the list of themes emerged after the data were coded separately by two researchers. The data were analyzed in accordance with the code and theme list after the final form of the code and theme list was collected. Students' opinions were collected under different codes, and the forms that emerged from the integration of these codes were determined as themes by the researchers. The consistency of the codes used independently by the researchers was determined by marking "agree" or "disagree". The cases in which the researchers used the same code for the students' statements were marked as an agreement, and the cases where they used different codes were marked as a disagreement. The inter-rater reliability was calculated using the formula $[\text{Agreement} / (\text{Agreement} + \text{Disagreement}) \times 100]$ (Miles & Huberman, 1994) as 89%.

3 Results

The findings obtained for the research problems addressed in the study are presented under two headings as quantitative and qualitative results.

3.1 Quantitative results

The descriptive statistics of reflective thinking and computational thinking scores in relation to gender are given in Table 2.

Without considering gender interaction, the change in reflective thinking and computational thinking abilities was examined. The results showed that there was a significant difference in reflective thinking scores for the pre-test ($M=2.240$, $SD=0.650$) and post-test ($M=2.709$, $SD=0.743$); $t(523)=32.163$, $p=.000$. For the level of computational thinking, there was a significant difference between the pre-test ($M=3.788$, $SD=0.691$) and post-test ($M=4.070$, $SD=0.591$) scores; $t(523)=44.081$, $p=.000$.

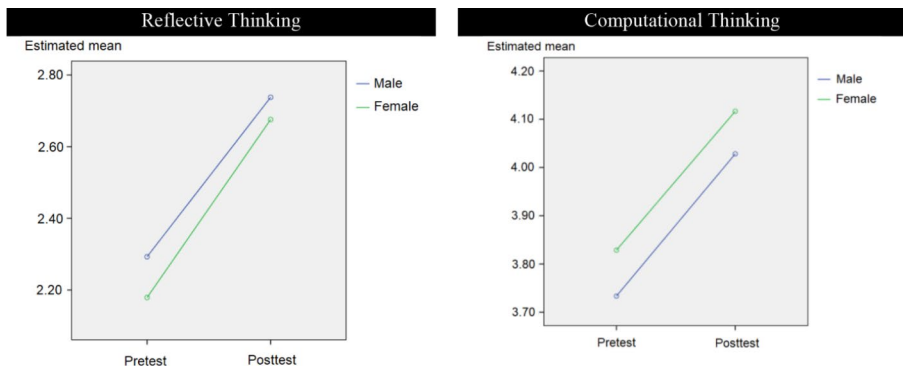


Fig. 9 The gender difference in reflective and computational thinking scores

The repeated measures ANOVA was conducted to compare the effect of Scratch programming instruction on female and male students' reflective and computational thinking. Box's M test was used for the assumption of equality variance-covariance matrices (Weinfurt, 2000). For the reflective thinking data, the Box's M value of 5.332 was interpreted as non-significant ($p < .05$) in conjunction with a p-value of 0.150. Similarly, a non-significant result was obtained for computational thinking scores, so the Box's M value of 1.713 was found in conjunction with a p value of 0.636. The gender difference in reflective and computational thinking scores is interpreted in Fig. 9.

The results showed that there was no significant effect of Scratch programming instruction on the reflective thinking skills of the female and male students, Wilks' Lambda=0.994, $F(1, 522)=3.156$, $p=.076$. This result indicates that the reflective thinking scale scores of the female and male students were not significantly differ depending on Scratch programming instruction.

Similar results were obtained for the computational thinking scale scores of the female and male students. It was found that there was no significant effect of Scratch programming instruction on computational thinking levels of female and male students, Wilks' Lambda=0.999, $F(1, 522)=0.266$, $p=.607$. These results state that the computational thinking scale scores of female and male students do not significantly differ depending on Scratch programming instruction.

3.2 Qualitative results

In this study, the effects of Scratch instruction on mathematical learning were reported through the written opinion forms presented to the students. Students were asked to what extent Scratch affected their concentration, participation, enjoyment and motivation in mathematics class. 87% of students reported that Scratch increased their concentration. The same opinion was supported by the opinions of 6 teachers among 8 teachers. About 74% of students stated that it increased their prompting to participate in class, 69% of them increased their enjoyment in class and 89% of them increased their motivation to attend class. At the same time, 4% of the students have a negative opinion about Scratch instruction. It was found that the majority of these

opinions included negative thoughts about technology usage. When examining the question of how you think Scratch affects your different ways of thinking, the codes that exceed 50% in student opinions are as follows: “improves my problem solving” (68%), “improves my analytical thinking” (62%) and “improves my critical thinking” (57%). The teachers’ opinions paralleled those of the students. The themes that emerged from the teachers’ opinions were critical thinking, holistic thinking, analytical thinking, problem solving, and computational thinking. The student codes had an initial “S” followed by their classroom and an ID number. For example, S5A14 identified the 14th student in classroom 5 A. Teacher codes began with an initial “T” followed by an ID number from 1 to 8 (e.g., T4). The examples of student and teacher opinions are listed below.

“Since I learned how to code, I realize that I can think differently. I can look at problems from a different perspective.” S6B17.

“I liked coding. It is exciting. I think I can bring different criticisms to situations by then.” S5F12.

“I think that different thinking styles of the students developed in this process. They have learned to look at problems in more detail, and their analytical thinking has also improved.” T4.

In general, the opinions of both teachers and students represented that Scratch instruction positively impacted the mathematical learning.

4 Discussion

Programming education has become increasingly important in today’s world, as the value of programming has increased. Programming classes that were once taught only in colleges and high schools are now taught in primary and elementary schools, and even preschoolers are being taught. In the teaching process, it is not enough for the student to have knowledge but individuals who can establish cause-effect relationships (Lind, 1998; Brewer, 2007), who are creative, and who can easily adapt to innovations (Rowe, 2007) are targeted. Similarly, in mathematics education, it is not enough to have mathematics knowledge only; it is aimed at educating individuals who can apply the knowledge they have learned (Demirel & Yağcı, 2012; Witrock, 1978), have problem solving skills (Clements & Gullo, 1984; Korkmaz et al., 2015), who are successful in communication and have positive attitudes toward mathematics (Hwa, 2018; Tutak, 2008). Technological materials have an active role in concretizing the abstract concepts of mathematics (Gelibolu, 2009; Kaput, 1999; Kieran, 1992; Taşlıbeyaz, 2010; Verbruggen et al., 2021). Mathematical softwares enable modeling (Aksakal et al., 2015) and understanding the problem solving process (Clements & Gullo, 1984; Korkmaz et al., 2015), by concretizing multiple representations such as numerical, algebraic, graphical (Azuma, 1997). It helps and supports the ways for thinking in different solutions. It is depicted that students who receive programming instruction effectively acquire higher order thinking skills such as mathematical thinking, systematic thinking and creative thinking beyond cooperative learning in the classroom.

With the change of what is expected of individuals in the new world, the production and transformation of knowledge is taking place faster and this situation makes it necessary for individuals to think differently, look at problems from different perspectives and change their problem solving strategies. For this reason, students and all members of society should develop problem solving and reflective thinking strategies in all extraordinary situations they encounter in their daily lives (Antonio, 2020; Göksün & Kurt, 2017).

In this sense, one of the most important skills a person should have, especially in recent years, is the ability to use reflective thinking to solve problems (Aras et al., 2022; Syamsuddin, 2020). Computational thinking, on the other hand, according to ISTE (2015), is a problem-solving strategy that strengthens the integration of technology and thinking. According to Wing (2008), computational thinking is a type of analytical thinking. Computational thinking according to Curzon (2015) is the process of individuals solving problems. Problem solving, reflective thinking, and computational thinking skills are examples of talents that can be fostered through the coding process. While coding improves an individual's mathematical skills, the individual also learns to design learning strategies and projects related to problem solving and to make connections between mathematical ideas during the coding process. This, in turn, can enable the development of skills deemed important for individual development in mathematics education. Based on the findings in the literature, the development of reflective thinking for problem solving and computational thinking skills of elementary students who received Scratch instruction, as well as changes in the mathematics learning environment, were examined with student and teacher opinions.

For problem solving, there was a significant difference between students' pretest and posttest reflective thinking scores. This finding can be interpreted to mean that Scratch instruction is effective in increasing students' reflective thinking skills in relation to problem solving. There are similar findings in the problem solving literature (Dasso et al., 2005; Kim et al., 2019; Papert, 1993). This result is also consistent with the nature of programming. Programming can also improve the ability to find a solution to a problem and the ability to analyze, as it is necessary to break the problem down into sub-sets and create a generalisable basic solution (Saeli et al., 2011). Writing program code is similar to the general problem solving process in that it often uses similar steps.

When the computational thinking results were analyzed, it was found that students' pre-test and post-test scores differed significantly, suggesting that Scratch instruction is effective in improving students' computational thinking skills. This is also consistent with the findings of many researchers that programming instruction improves computational thinking skill (Allsop, 2015; Angeli et al., 2016; Kalelioğlu & Gülbahar, 2014b; ScratchEd Team, 2011; Settle & Perkovic, 2010; Wing, 2006).

The results showed that scores for reflective thinking towards problem solving and computational thinking did not vary by gender. Some studies of gender in the literature show that female students have begun to engage more actively in programming and coding (Colley & Comber, 2003; Fraillon et al., 2014), and it is noted that there is a significant difference in favor of female students, especially in application scores (Durak & Guyer, 2019), with female students learning algorithmic concepts better

(Hsu, 2014). In our study, there was no gender difference in scores for reflective thinking for problem solving and computational thinking. Both genders are enthusiastic about Scratch practices, and successful implementation may serve as a predictor of this outcome.

When opinions were evaluated in general, it was found that teachers and students believed that Scratch instruction positively influenced mathematics learning. Similar results have been obtained in some studies on technology and computer use in mathematics education (Corbalan et al., 2010; Kutluca & Birgin, 2007; Yıldırım, 2011). In the study of Yıldız et al., (2012), it was found that technology-based problem solving increases interest and makes students feel active, with similar results. Scratch enables making mathematics concrete instead of abstract (Ersoy, 2003), adding visuality to the lesson (Genç-Çelik & Şengül, 2005), and creating an educational work environment that students enjoy (Hill, 2015) and creating a sense of purpose for the subject (Tatar et al., 2013) are expressed in both students' opinions and related literature. Students took a more active role in the activities and used Scratch in a more meaningful way as a result of this process, as they generated the codes themselves and set up the required algorithms.

5 Conclusion

In short, Scratch developed reflective thinking skills for problem solving and computational thinking skills of elementary school students. In addition, Scratch created positive reflections in the mathematics learning environment. Based on this study, the effects of teaching can be investigated by designing applications for teaching mathematics with Scratch. In-service training could be conducted for mathematics teachers and they could be asked to develop instructional activities using Scratch during these trainings. Scratch use has been found to have a positive effect, but it is not known how long these effects of Scratch last in the long run. Therefore, the durability of the effect on these thinking skills can be investigated in future studies. Scratch applications can be offered as an elective in the curriculum to enable prospective mathematics teachers studying in faculties of education to use this software more accurately and effectively. The close connection between the content of Scratch instruction and the problem-solving process in the mathematics classroom in this study can be considered in the context of the mathematics standards (e.g., Common Core State Standards for Mathematics) during their revision processes.

6 Limitation

Limitations of the study include the lack of a control/comparison group and measurement of ability level. There was also no direct measurement of mathematics or programming skills. Considering the limitations of this study, a more comprehensive study may be conducted in future studies.

Declarations

Conflict interests The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this manuscript. All data generated or analyzed during this study are included in this published article.

References

- AbdulSamad, U., & Romli, R. (2022). A Comparison of Block-Based Programming Platforms for Learning Programming and Creating Simple Application. In *International Conference of Reliable Information and Communication Technology* (pp. 630–640). Springer, Cham
- Aksakal, M., Karataş, A., & Şimşek, C. L. (2015). The effect of a laboratory environment enriched with models on academic success within the scope of teaching the subject of meiosis. *Pamukkale University Journal of Education*, 37, 49–60. <https://doi.org/10.9779/puje621>
- Akşit, O. (2018). *Enhancing science learning through computational thinking and modeling in middle school classrooms: A mixed methods study*. Dissertation, North Carolina State University, Raleigh, NC
- Alakoç, Z. (2003). Technological modern teaching approaches in mathematics teaching. *The Turkish Online Journal of Educational Technology*, 2(1), 43–49
- Allsop, Y. (2015). ICT'den kodlamaya: İngiltere'de teknoloji eğitimi. *Eğitim Teknolojileri Zirvesi*, (s. 303–308). Ankara
- Altinok, N., Angrist, N., & Patrinos, H. A. (2018). Global data set on education quality (1965–2015). *World Bank Policy Research Working Paper*, (8314)
- Alvarado, C., Lee, C. B., & Gillespie, G. (2014, March). New CS1 pedagogies and curriculum, the same success factors? In *Proceedings of the 45th ACM technical symposium on computer science education* (pp. 379–384) New York, NY, USA: ACM. <https://doi.org/10.1145/2538862.2538897>
- Angeli, C., Voogt, J., Fluck, A., Webb, M., Cox, M., Malyn-Smith, J., & Zagami, J. (2016). A K-6 computational thinking curriculum framework: Implications for teacher knowledge. *Educational Technology & Society*, 19(3), 47–57
- Antonio, R. P. (2020). Developing Students' Reflective Thinking Skills in a Metacognitive and Argument-Driven Learning Environment. *International Journal of Research in Education and Science*, 6(3), 467–483
- Aras, I., Pratiwi, E., Nanna, A. W. I., & Barumbun, M. (2022). Role of scaffolding for reflective thinking on the mathematical problem solving. *Aksioma: Jurnal Program Studi Pendidikan Matematika*, 11(1), 718–726
- Arfè, B., Vardanega, T., & Ronconi, L. (2020). The effects of coding on children's planning and inhibition skills. *Computers & Education*, 148, 103807
- Atmatzidou, S., & Demetriadis, S. (2016). Advancing students' computational thinking skills through educational robotics: A study on age and gender relevant differences. *Robotics and Autonomous Systems*, 75, 661–670. <https://doi.org/10.1016/j.robot.2015.10.008>
- Azuma, R. (1997). A survey of augmented reality. *Presence: Teleoperators and Virtual Environments*, 6(4), 355–385. <https://doi.org/10.1162/pres.1997.6.4.355>
- Bråting, K., & Kilhamn, C. (2021). Exploring the intersection of algebraic and computational thinking. *Mathematical Thinking and Learning*, 23(2), 170–185 <https://doi.org/10.1080/10986065.2020.1779012>
- Brewer, J. A. (2007). *Introduction to early childhood education: Preschool through primary grades* (6th ed.). New York: Allyn & Bacon
- Carter, J., & Jenkins, T. (1999, June). Gender and programming: What's going on?. In *Proceedings of the 4th annual SIGCSE/SIGCUE ITiCSE Conference on Innovation and Technology in Computer Science Education* (pp. 1–4)
- Celen, Y. (2020). Student Opinions on the Use of Geogebra Software in Mathematics Teaching. *Turkish Online Journal of Educational Technology-TOJET*, 19(4), 84–88
- Cengiz, C. (2014). The effect of reflective journals kept by pre-service science teachers on reflective thinking and achievement in general chemistry laboratory. Unpublished PhD Thesis. Karadeniz Technical University, Trabzon

- Chevallard, Y., & Bosch, M. (2020). Didactic transposition in mathematics education. *Encyclopedia of mathematics education*, 214–218
- Clements, D. H., & Gullo, D. F. (1984). Effects of computer programming on young children's cognition. *Journal of Educational Psychology*, 76(6), 1051–1058. <https://doi.org/10.1037/0022-0663.76.6.1051>
- Colley, A., & Comber, C. (2003). Age and gender differences in computer use and attitudes among secondary school students: what has changed? *Educational Research*, 45(2), 155–165. <https://doi.org/10.1080/0013188032000103235>
- Coravu, L., Marian, M., & Ganea, E. (2015, September). Scratch and recreational coding for kids. In *2015 14th RoEduNet International Conference-Networking in Education and Research (RoEduNet NER)* (pp. 85–89). IEEE. <https://doi.org/10.1109/RoEduNet.2015.7311973>
- Corbalan, G., Paas, F., & Cuypers, H. (2010). Computer-based feedback in linear algebra: Effects on transfer performance and motivation. *Computers & Education*, 55(2), 692–703. <https://doi.org/10.1016/j.compedu.2010.03.002>
- Creswell, J. W., & Clark, V. L. P. (2018). *Karma Yöntem Araştırmaları: Tasarımı ve Yürütülmesi* (Çev. Ed. Y. Dede ve S. B. Demir). Ankara: Anı Yayıncılık
- Curzon, P. (2015). *Computational thinking: Searching to speak*. Available at: <http://teachinglondoncomputing.org/free-workshops/computational-thinkingsearching-to-speak/>
- Çatlak, Ş., Tekdal, M., & Baz, F. (2015). The status of teaching programming with Scratch: a document review work. *Journal of Instructional Technologies & Teacher Education*, 4(3), 13–25
- Çiftci, S., & Bildiren, A. (2020). The effect of coding courses on the cognitive abilities and problem-solving skills of preschool children. *Computer science education*, 30(1), 3–21
- Dasso, A., Funes, A., Riesco, D., Montejano, G., Peralta, M., & Salgado, C. (2005). Teaching programming. *Proceedings of JEITICS*, Educación en Informática y TICs en Argentina, 183–186
- Davadas, S. D., & Lay, Y. F. (2020). Contributing factors of secondary students' attitude towards mathematics. *European Journal of Educational Research*, 9(2), 489–498. <https://doi.org/10.12973/eu-jer.9.2.489>
- Demirel, M., & Yağcı, E. (2012). Perceptions of primary school teacher candidates about lifelong learning. *Hacettepe University Journal of Education*, 1, 100–111
- Dewey, J. (1933). *How we think: A restatement of the relation of reflective thinking to the educative process*. Boston: D. C. Heath
- Domínguez, A., Saenz-de-Navarrete, J., De-Marcos, L., Fernández-Sanz, L., Pagés, C., & Martínez-Herráiz, J. J. (2013). Gamifying learning experiences: Practical implications and outcomes. *Computers & Education*, 63, 380–392. <https://doi.org/10.1016/j.compedu.2012.12.020>
- Du, J., Wimmer, H., & Rada, R. (2016). “Hour of Code”: Can it change students' attitudes toward programming? *Journal of Information Technology Education: Innovations in Practice*, 15, 53–73. <https://doi.org/10.28945/3421>
- Durak, H. Y., & Guyer, T. (2019). Programming with Scratch in primary school, indicators related to effectiveness of education process and analysis of these indicators in terms of various variables. *Gifted Education International*, 35(3), 237–258. <https://doi.org/10.1177/0261429419854223>
- Elia, I., van den Heuvel-Panhuizen, M., & Kolovou, A. (2009). Exploring strategy use and strategy flexibility in non-routine problem solving by primary school high achievers in mathematics. *Zdm Mathematics Education*, 41(5), 605–618. <https://doi.org/10.1007/s11858-009-0184-6>
- Erol, O., & Çırak, N. S. (2022). The effect of a programming tool scratch on the problem-solving skills of middle school students. *Education and Information Technologies*, 27(3), 4065–4086
- Ersoy, Y. (2003). Teknoloji destekli matematik eğitimi 1: Gelişmeler, politikalar ve stratejiler. *İlköğretim-Online*, 2(1), 18–27
- Fesakis, G., & Serafeim, K. (2009). Influence of the familiarization with “Scratch” on future teachers' opinions and attitudes about programming and ICT in education. *ACM SIGCSE Bulletin*, 41(3), 258–262. <https://doi.org/10.1145/1595496.1562957>
- Ferracane, M. F. (2020). Redesigning traditional education. *Redesigning Organizations* (pp. 329–343). Cham: Springer
- Fichtner, B. (2005). Reflective learning-Problems and Questions Concerning a Current Contextualization of the Vygotskian Approach. In (Eds.) Hoffmann, M. H.G., Lenhard, J. and Seeger, F. *Activity and Sign Grounding Mathematics Education* (179–190). US: Springer
- Flanagan, S. (2015). Introduce programming in a fun, creative way. *Tech Directions*, 74(6), 18

- Fraillon, J., Ainley, J., Schulz, W., Friedman, T., & Gebhardt, E. (2014). *Preparing for life in a digital age: The IEA International Computer and Information Literacy Study international report* (p. 308). Springer Nature
- Furber, S. (2012). *Shut down or restart? The way forward for computing in UK schools*. London: The Royal Society
- Garner, S. (2003). Learning resources and tools to aid novices learn programming. In *Informing science & information technology education joint conference (INSITE)* (pp. 213–222)
- Gelibolu, M. F. (2009). Evaluation of application of computer assisted logic instruction materials developed in realistic mathematics approach in 9th grade mathematics lessons. Unpublished Masters Thesis. Ege University, Izmir
- Genç Çelik, N., & Şengül, S. (2005). The effect of mastery learning method on the achievement and retention level of 6th grade primary school students in the mathematics class. *Journal of Uludağ University Faculty of Education*, 18(1), 107–122
- Gonzalez, C. (Ed.). (2012). *Student Usability in Educational Software and Games: Improving Experiences: Improving Experiences*. IGI Global
- Göksün, D. O., & Kurt, A. A. (2017). The relationship between pre-service teachers' use of 21st century learner skills and 21st century teacher skills. *Education and Science*, 42(190), 107–130. <https://doi.org/10.15390/eb.2017.7089>
- Havenga, M., Breed, B., Mentz, E., Govender, D., Govender, I., Dignum, F., & Dignum, V. (2013). Meta-cognitive and problem-solving skills to promote self-directed learning in computer programming: Teachers' experiences. *SA-eDUC Journal*, 10(2), 1–14
- Hill, C. (2015). Programming environments for children: Creating a language that grows with you. Unpublished PhD Thesis, UC Santa Barbara
- Howell, D. C. (2002). *Statistical Methods for Psychology* (5th ed.). Pacific Grove CA: Duxbury
- Hsu, H. M. J. (2014, February). Gender differences in Scratch Game design. In *2014 International Conference on Information, Business and Education Technology (ICIBET 2014)* (pp. 100–103). Atlantis Press
- Hsu, T. C., Chang, S. C., & Hung, Y. T. (2018). How to learn and how to teach computational thinking: Suggestions based on a review of the literature. *Computers & Education*, 126, 296–310. <https://doi.org/10.1016/j.compedu.2018.07.004>
- Huck, S. W. (2012). *Reading statistics and research* (6th Edition). Boston, MA: Pearson
- Hwa, S. P. (2018). Pedagogical change in mathematics learning: Harnessing the power of digital game-based learning. *Journal of Educational Technology & Society*, 21(4), 259–276
- International Society for Technology in Education [ISTE] (2007). *ISTE Standards for Students 2007*. Retrieved on December 04, 2020, from <https://www.iste.org/standards/for-students>
- International Society for Technology in Education [ISTE] (2011). *Operational definition of computational thinking for K-12 education*. Retrieved on December 25, 2020 from <https://cdn.iste.org/www-root/ct-documents/computational-thinking-operational-definition-flyer.pdf>
- International Society for Technology in Education [ISTE] (2015). *Computational thinking for all*. Retrieved on December 14, 2020 from <https://www.iste.org/explore/Solutions/Computational-thinking-for-all>
- International Society for Technology in Education [ISTE] (2016). *ISTE Standards for Students 2016*. Retrieved on December 05, 2020 from <https://www.iste.org/standards/downloads>
- Kalelioğlu, F., & Gülbahar, Y. (2014a). The effect of instructional techniques on critical thinking and critical thinking dispositions in online discussion. *Journal of Educational Technology & Society*, 17(1), 248–258
- Kalelioğlu, F., & Gülbahar, Y. (2014b). The effects of teaching programming via scratch on problem solving skills: A discussion from learners' perspective. *Informatics in Education*, 13(1), 33–50. <https://doi.org/10.15388/infedu.2014.03>
- Kalogiannakis, M., Ampartzaki, M., Papadakis, S., & Skaraki, E. (2018). Teaching natural science concepts to young children with mobile devices and hands-on activities. A case study. *International Journal of Teaching and Case Studies*, 9(2), 171–183. <https://doi.org/10.1504/ijtes.2018.090965>
- Kaput, J. J. (1999). Teaching and learning a new algebra. In T. Romberg, & E. Fennema (Eds.), *Mathematics classrooms that promote understanding* (pp. 133–155). Mahwah, NJ: Lawrence Erlbaum Associates, Inc
- Kenedi, A. K., Helsa, Y., Ariani, Y., Zainil, M., & Hendri, S. (2019). Mathematical Connection of Elementary School Students to Solve Mathematical Problems. *Journal on Mathematics Education*, 10(1), 69–80

- Kızılkaya, G., & Aşkar, P. (2009). The development of a reflective thinking skill scale towards problem solving. *Education and Science*, 34(154), 82–92
- Kieran, C. (1992). The learning and teaching of school algebra. In D. Grouws (Ed.), *Handbook of Research in Mathematics Teaching and Learning* (pp. 390–419). New York: Macmillan
- Kim, S., Choe, I., & Kaufman, J. C. (2019). The development and evaluation of the effect of creative problem-solving program on young children's creativity and character. *Thinking Skills and Creativity*, 33, 100590
- Kobsiripat, W. (2015). Effects of the media to promote the scratch programming capabilities creativity of elementary school students. *Procedia-Social and Behavioral Sciences*, 174, 227–232. <https://doi.org/10.1016/j.sbspro.2015.01.651>
- Korkmaz, Ö., Çakır, R., & Özden, M. Y. (2015). Computational thinking levels scale (CTLs) adaptation for secondary school level. *Gazi Journal of Educational Sciences*, 1(2), 143–162
- Kukul, V. (2013). Eğitsel Dijital Oyunlar. Pegem A Akademi Yayıncılık, 320 s, Ankara
- Kuo, M., Barnes, M., & Jordan, C. (2019). Do experiences with nature promote learning? Converging evidence of a cause-and-effect relationship. *Frontiers in Psychology*, 10, 305. <https://doi.org/10.3389/fpsyg.2019.00305>
- Kutluca, T., & Birgin, O. (2007). Evaluation of prospective mathematics teachers' views about computer assisted teaching material developed in the linear equation topic. *Gazi University Journal of Gazi Educational Faculty*, 2, 81–97
- Lewis, C. M. (2010, March). How programming environment shapes perception, learning and goals: Logo vs. Scratch. In *Proceedings of the 41st ACM technical symposium on computer science education* (pp. 346–350)
- Lewis, A., & Smith, D. (1993). Defining higher order thinking: Theory into practice. Teaching for Higher Order Thinking, 131–137
- Lind, K. K. (1998). *Science in early childhood developing and acquiring fundamental concepts and skills* (pp. 3–18). Washington DC: National Science Foundation
- Linn, M. C. (1985). Fostering equitable consequences from computer learning environments. *Sex Roles*, 13(3–4), 229–240
- Liu, Y. C., Huang, T. H., & Sung, C. L. (2021). The determinants of impact of personal traits on computational thinking with programming instruction. *Interactive Learning Environments*, 1–15. <https://doi.org/10.1080/10494820.2021.1983610>
- Lockheed, M. E. (1985). Women, girls, and computers: A first look at the evidence. *Sex Roles*, 13(3–4), 115–122
- Maloney, J., Resnick, M., Rusk, N., Peppler, K. A., & Kafai, Y. B. (2008, June). Media designs with Scratch: What urban youth can learn about programming in a computer clubhouse. In *Proceedings of the 8th International conference for the learning sciences-Volume 3* (pp. 81–82)
- Maloney, J., Resnick, M., Rusk, N., Silverman, B., & Eastmond, E. (2010). The Scratch programming language and environment. *ACM Transactions on Computing Education (TOCE)*, 10(4), 1–15. <https://doi.org/10.1145/1868358.1868363>
- Mangaroska, K., Sharma, K., Gašević, D., & Giannakos, M. (2022). Exploring students' cognitive and affective states during problem solving through multimodal data: Lessons learned from a programming activity. *Journal of Computer Assisted Learning*, 38(1), 40–59
- Margolis, J., & Goode, J. (2016). Ten lessons for computer science for all. *ACM Inroads*, 7(4), 52–56. <https://doi.org/10.1145/2988236>
- Mayer-Smith, J., Pedretti, E., & Woodrow, J. (2000). Closing of the gender gap in technology enriched science education: a case study. *Computers & Education*, 35(1), 51–63. [https://doi.org/10.1016/S0360-1315\(00\)00018-X](https://doi.org/10.1016/S0360-1315(00)00018-X)
- Miles, M. B., & Huberman, A. M. (1994). *An expanded sourcebook: Qualitative data analysis*. London: Sage
- Ministry of National Education [MoNE]. (2018a). *Information Technologies and Software National Curriculum (for grades 5 and 6)*. Ankara: Board of Education
- Ministry of National Education [MoNE]. (2018b). *Mathematics National Curriculum (for grades 1, 2, 3, 4, 5, 6, 7 and 8)*. Ankara: Board of Education
- Olkun, S., & Toluk Uçar, Z. (2006). *İlköğretimde matematik öğretimine çağdaş yaklaşımlar*. Ankara: Ekinoks Yayınları
- Organisation for Economic Co-operation and Development [OECD]. (2011). *Better Policies to Support Eco-innovation, OECD Studies on Environmental Innovation*. Paris: OECD Publishing. <https://doi.org/10.1787/9789264096684-en>

- Papert, S. (1980). *Mindstorms: Children, computers, and powerful ideas*. New York: Inc., Publishers
- Papert, S. (1993). *The children's machine: Rethinking school in the age of the computer*. NY: Basic Books
- Polya, G. (1973). *How to solve it: A new aspect of mathematical method* (2nd ed.). Princeton, New Jersey: Princeton University Press
- Quaiser-Pohl, C., Geiser, C., & Lehmann, W. (2006). The relationship between computer-game preference, gender, and mental-rotation ability. *Personality and Individual Differences*, 40(3), 609–619. <https://doi.org/10.1016/j.paid.2005.07.015>
- Resnick, M. (2013). Learn to code, code to learn: How programming prepares kids for more than math. EdSurge (May 8, 2013)
- Resnick, M., Maloney, J., Monroy-Hernández, A., Rusk, N., Eastmond, E., Brennan, K., & Kafai, Y. (2009). Scratch: programming for all. *Communications of the ACM*, 52(11), 60–67
- Resnick, M., & Silverman, B. (2005, June). Some reflections on designing construction kits for kids. In *Proceedings of the 2005 conference on Interaction design and children* (pp. 117–122)
- Rodríguez-Martínez, J. A., González-Calero, J. A., & Sáez-López, J. M. (2020). Computational thinking and mathematics using Scratch: an experiment with sixth-grade students. *Interactive Learning Environments*, 28(3), 316–327
- Rowe, A. J. (2007). Yaratıcı Zekâ. Prestij Yayınları, 220 s., İstanbul
- Saeli, M., Perrenet, J., Jochems, W. M. G., & Zwaneveld, B. (2011). Teaching programming in secondary school: A pedagogical content knowledge perspective. *Informatics in Education*, 10(1), 73–88
- Scratch About (2021). Retrieved on August 25, 2021 from <http://scratch.mit.edu/about/>
- Scratch Stats (2021). Retrieved on August 25, 2021 from <https://scratch.mit.edu/statistics/>
- ScratchEd Team. (2011). *Creative computing: A design-based introduction to computational thinking*. MIT Media Lab
- Settle, A., & Perkovic, L. (2010). Computational thinking across the curriculum: A conceptual framework. Technical Report, DePaul University College of Computing and Digital Media. Retrieved on January 11, 2021, from <https://via.library.depaul.edu/tr/13>
- Shashaani, L., & Khalili, A. (2001). Gender and computers: Similarities and differences in Iranian college students' attitudes toward computers. *Computers & Education*, 37(3–4), 363–375. [https://doi.org/10.1016/S0360-1315\(01\)00059-8](https://doi.org/10.1016/S0360-1315(01)00059-8)
- Shanta, S., & Wells, J. G. (2020). T/E design based learning: assessing student critical thinking and problem solving abilities. *International Journal of Technology and Design Education*, 1–19
- Shermis, S. S. (1992). *Critical Thinking: Helping Students Learn Reflectively*. Bloomington: EDINFO Press
- Sholihah, T. M., & Lastariwati, B. (2020). Problem Based Learning to Increase Competence of Critical Thinking and Problem Solving. *Journal of Education and Learning (EduLearn)*, 14(1), 148–154
- Sivilotti, P. A., & Laugel, S. A. (2008). Scratching the surface of advanced topics in software engineering: a workshop module for middle school students. *ACM SIGCSE Bulletin*, 40(1), 291–295. <https://doi.org/10.1145/1352322.1352235>
- Sun, L., Hu, L., & Zhou, D. (2021). Improving 7th-graders' computational thinking skills through unplugged programming activities: A study on the influence of multiple factors. *Thinking Skills and Creativity*, 42, 100926
- Syamsuddin, A. (2020). Describing taxonomy of reflective thinking for field dependent-prospective mathematics teacher in solving mathematics problem. *International Journal of Scientific & Technology Research*, 9(3), 4418–4421
- Taşlıbeyaz, E. (2010). A case study research intended for mathematics perception at computer aided mathematics teaching for secondary students: Application of high school 3rd year. Unpublished Masters Thesis, Atatürk University, Erzurum
- Tatar, E., Zengin, Y., & Kağızmanlı, T. B. (2013). The use of dynamic mathematics software and interactive white board technology in mathematics teaching. *Turkish Journal of Computer and Mathematics Education*, 4(2), 104–123
- Taylor, M., Harlow, A., & Forret, M. (2010). Using a computer programming environment and an interactive whiteboard to investigate some mathematical thinking. *Procedia-Social and Behavioral Sciences*, 8, 561–570. <https://doi.org/10.1016/j.sbspro.2010.12.078>
- Tutak, T. (2008). The effects of using concrete materials and dynamic geometry software on students' cognitive learning, attitudes, and understanding levels of Van Hiele geometry. Unpublished PhD Thesis, Karadeniz Technical University, Trabzon
- Ünver, G. (2011). Eğitimde yeni yönelimler (5. Baskı). Ö. Demirel (Ed.), Yansıtıcı düşünme (137–148). Ankara: Pegem Akademi Yayınlar#305

- Üstünoğlu, E. (2006). Üst düzey düşünme becerilerini geliştirmede bilişsel soruların rolü. *Çağdaş Eğitim Dergisi*, 331, 17–24
- Verbruggen, S., Depaepe, F., & Torbeyns, J. (2021). Effectiveness of educational technology in early mathematics education: A systematic literature review. *International Journal of Child-Computer Interaction*, 27, 100220
- Wei, X., Lin, L., Meng, N., Tan, W., & Kong, S. C. (2021). The effectiveness of partial pair programming on elementary school students' computational thinking skills and self-efficacy. *Computers & Education*, 160, 104023
- Weinfurt, K. P. (2000). Repeated measures analysis: ANOVA, MANOVA, and HLM. *Yarnold (Eds.), Reading and understanding MORE multivariate statistics* (pp. 317–361). American Psychological Association. *Grimm*
- Wells, P., De Lange, P., & Fieger, P. (2008). Integrating a virtual learning environment into a second-year accounting course: determinants of overall student perception. *Accounting & Finance*, 48(3), 503–518. <https://doi.org/10.1111/j.1467-629X.2007.00249.x>
- Weng, X., Ng, O. L., Cui, Z., & Leung, S. (2022). Creativity Development with Problem-Based Digital Making and Block-Based Programming for Science, Technology, Engineering, Arts, and Mathematics Learning in Middle School Contexts. *Journal of Educational Computing Research*, 1–25. <https://doi.org/10.1177/07356331221115661>
- Wing, J. M. (2006). Computational Thinking. *Communications of the ACM*, 49(3), 33–35
- Wing, J. M. (2008). Computational thinking and thinking about computing. *Philosophical Transactions of the Royal Society A: Mathematical Physical and Engineering Sciences*, 366(1881), 3717–3725. <https://doi.org/10.1098/rsta.2008.0118>
- Wing, J. M. (2014). Computational thinking benefits society. *40th Anniversary Blog of Social Issues in Computing*, 2014, 26
- Witrock, M. C. (1978). The cognitive movement in instruction. *Education Psychologist*, 13, 15–29. <https://doi.org/10.1080/00461527809529192>
- Yıldırım, I. (2011). In the framework of technologically supported math education, to explore the usage of alternative measurement materials. Unpublished Masters Thesis, Gaziantep University, Gaziantep
- Yıldız, A., Baltacı, S., & Aktümen, M. (2012). Elementary mathematics teacher candidates' processes of problem solving about three-dimensional objects through dynamic mathematics software. *Kastamonu Eğitim Dergisi*, 20(2), 592–604
- Zengin, Y. (2019). Development of mathematical connection skills in a dynamic learning environment. *Education and Information Technologies*, 24(3), 2175–2194
- Zuckerman, O., Blau, I., & Monroy-Hernández, A. (2009). Children's participation patterns in online communities. *Interdisciplinary Journal of E-Learning and Learning Objects*, 5(1), 263–274

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.